

Aquí tienes el análisis técnico detallado, enfocado a preguntas de examen tipo test, desglosando la lógica de hilos, concurrencia y configuración de Spring que aparece en tu código.

1. Configuración del Pool de Hilos (**AsyncConfig.java**)

Esta clase es fundamental para entender cómo se comportan los hilos antes de que el sistema se sature. Aquí se definen los **ThreadPoolTaskExecutor**.

Conceptos Clave para el Examen:

- **@EnableAsync**: Habilita la capacidad de Spring para detectar la anotación **@Async**. Sin esto, los métodos se ejecutarían de forma síncrona (secuencial) aunque tengan la etiqueta.
- Ciclo de vida de un Pool (Core vs Queue vs Max):
Esta es la pregunta típica de examen "trampa". Analicemos el bean taskExecutor:
 - **CorePoolSize(3)**: Hilos mínimos siempre listos.
 - **QueueCapacity(20)**: Cola de espera.
 - **MaxPoolSize(6)**: Hilos máximos permitidos.
- **Comportamiento lógico (Algoritmo de decisión)**:
 - Llegan tareas. Si hay menos de 3 tareas corriendo -> Se usan los hilos del Core.
 - Si llegan la 4ª, 5ª... tarea -> **NO** se crean más hilos inmediatamente. Se encolan en la **Queue** (hasta 20).
 - Solo si la cola se llena (20 tareas en espera + 3 ejecutándose) -> Se crean nuevos hilos hasta llegar al **MaxPoolSize** (6).
 - Si se supera el Max (6 hilos + 20 en cola) -> **RejectionExecutionException** (o la política de rechazo configurada).
- **Análisis del bean **threadsJurado****:
 - Diseñado para alta carga (Core 10, Max 15).
 - Cola enorme (1000). Es muy difícil que este pool cree más hilos que el **Core** (10) a menos que lleguen más de 1010 peticiones simultáneas.

2. El Problema del Proxy y **@Async** (**PeliculaService.java**)

Aquí hay un punto crítico sobre cómo Spring gestiona la asincronía que suele caer en exámenes.

Invocación Correcta vs Incorrecta (Self-Invocation)

Caso Correcto (**procesarPeliculasAsync**):

Java

```
// PeliculaService.java
```

```
var t1 = this.tareaAsync.tareaLenta2Async("...");
```

- Llama a un método **@Async** que está en **otra clase** (**TareaAsync**). Spring intercepta la llamada mediante un Proxy e inyecta la ejecución en un hilo separado. **Esto es asíncrono real.**

Caso Incorrecto / Trampa (**reproducirAsync**):

```
Java
// PeliculaService.java
public String reproducirAsync() {
    // ...
    var t1 = this.reproducir("..."); // LLAMADA INTERNA
    // ...
}
```

```
@Async("taskExecutor")
```

```
public CompletableFuture<String> reproducir(String titulo) { ... }
```

- **Análisis:** Al llamar a `this.reproducir()`, estás llamando al método directamente dentro de la misma instancia de la clase, saltándote el Proxy de Spring.
Resultado: La anotación `@Async` SE IGNORA. El código se ejecutará secuencialmente en el hilo principal (Main Thread). `t1` bloqueará hasta terminar, luego `t2`, etc.
 - **Nota:** Si en el examen te preguntan si `reproducirAsync` aprovecha el multicore, la respuesta es **NO**, debido a la "Self-invocation" de Spring AOP.

3. Sincronización y Race Conditions (**TareaAsync.java** - Método **votar**)

Este método es el ejemplo perfecto de concurrencia y seguridad de hilos (Thread Safety).

```
Java
@Async("threadsJurado")
public CompletableFuture<Void> votar(Map<String, Integer> votos, int juradoId) {
    // ...
    synchronized (votos) {
        votos.put(pelicula, votos.get(pelicula) + puntos);
    }
    // ...
}
```

Puntos de Análisis:

1. **Recurso Compartido:** El objeto `Map<String, Integer> votos` se crea en `PeliculaService` y se pasa por referencia a múltiples hilos (Jurados).
 2. **Race Condition (Condición de Carrera):** Si dos hilos intentan escribir en el mapa al mismo tiempo (ej: sumar puntos a "Avatar"), uno podría sobrescribir el cambio del otro, perdiendo votos.
 3. **Solución (**synchronized**):**
 - El bloque `synchronized (votos)` crea un **Monitor** o cerrojo (lock) sobre el objeto `votos`.
 - Solo **un hilo** puede entrar en ese bloque a la vez. Los demás hilos que intenten entrar se quedan en estado **BLOCKED** hasta que el primero salga.
 - **Impacto en rendimiento:** Reduce ligeramente el paralelismo (cuello de botella), pero garantiza la integridad de los datos.
-

4. Orquestación con **CompletableFuture**

En **PeliculaService** y **Controller**, se usa mucho esta clase. Es la evolución de **Future**.

- **CompletableFuture.completedFuture(...)**: Se usa para devolver un valor estático pero envuelto en una promesa, necesario para cumplir con la firma del método **@Async**.
- **CompletableFuture.allOf(t1, t2...).join()**:
 - **allOf**: Crea una nueva tarea que se completa solo cuando **todas** las tareas pasadas por parámetro terminan.
 - **join()**: Es un método **bloqueante**. El hilo principal (el del Controller/Service) se detiene aquí y espera a que terminen los hilos secundarios.
 - *Pregunta de examen*: ¿Por qué usamos **join()** y no **get()**? **join()** lanza excepciones no chequeadas (**CompletionException**), lo cual es más limpio en lambdas y streams que **get()**, que lanza excepciones chequeadas.

5. Lectura de Archivos Asíncrona (**importarPeliculas**)

Java

```
// PeliculaService.java
List<CompletableFuture<Void>> futures = new ArrayList<>();
// ... loop ...
futures.add(this.tareaAsync.importarCsvAsync(path));
// ...
CompletableFuture.allOf(futures.toArray(...)).join();
```

- **Patrón Fork/Join (manual)**:
 1. **Fork**: El bucle lanza múltiples tareas al **Executor**. No espera a que una termine para lanzar la siguiente.
 2. **Join**: La línea **allOf(...).join()** actúa como barrera de sincronización. Asegura que el método **importarPeliculas** no retorne hasta que todos los ficheros hayan sido procesados.

Resumen Rápido para el Test

Concepto	Comportamiento en tu código
@Async("nombre")	Envía la ejecución al ThreadPoolTaskExecutor con ese nombre bean.
ThreadPoolTaskExecutor	Gestiona los hilos. Si Core lleno -> Cola. Si Cola llena -> Max.

this.metodoAsync()	Trampa: Se ejecuta síncrono (bloqueante) en el mismo hilo.
bean.metodoAsync()	Se ejecuta asíncrono (multihilo) correctamente.
synchronized(obj)	Bloquea el objeto obj para evitar corrupción de datos en escritura concurrente.
CompletableFuture.join()	El hilo principal espera a que terminen los hilos asíncronos.
Thread.sleep()	Simula carga de CPU. Si está dentro de un synchronized , retiene el lock (malo). Si está fuera, solo pausa el hilo actual (bueno).

Posible Pregunta de Examen Práctica

Pregunta: Si lanzas el endpoint /oscar/100 (100 jurados) y tu configuración es Core=10, Max=15, Queue=1000. ¿Cuántos hilos se crean realmente?

Respuesta: Se crearán 10 hilos.

Razonamiento: Llegan 100 peticiones.

1. Las primeras 10 ocupan el **Core**.
2. Las siguientes 90 caben perfectamente en la **Queue** (capacidad 1000).
3. Como la cola no se ha llenado, el pool **no** necesita escalar hasta **Max (15)**. Se procesan con los 10 hilos reciclándolos a medida que terminan.